OFFLINE MODERATED LEARNING MANAGEMENT SYSTEM

CS4433: Software Engineering Individual Assignment National University of Lesotho Department of Mathematics and Computer Science

Student Name: Khotso Makoa **Student Number:** 202322652 **Submission Date:** 3 November 2026

GitHub Repository: https://github.com/makoakhotso-oss/CS4433-MODERATED-LMS-PROJECT

Abstract

This project involved developing an Offline Moderated Learning Management System (LMS) for a secondary school. The system manages courses, assignments, student submissions, marking, moderation and the release of results. It was developed using C++17, SQLite and CMake, following the Waterfall model with V-Model verification and validation. Four user roles are supported: Administrator, Lecturer, Student and Head of Department (HoD). The implementation includes secure authentication, role-based access, late penalties, individual extensions, resubmission and mark auditing. All 104 automated tests passed, with 96.7% service-layer line coverage. This report summarises the eight development phases, engineering decisions, testing evidence, design changes and remaining limitations.

1. Introduction

The project addresses challenges associated with manual or disconnected assignment management, including lost submissions, inconsistent late penalties, unclear marking histories and the release of marks before moderation.

The objective was to develop a secure, reliable and easy-to-use offline LMS suitable for a school with approximately 800 students and unreliable internet connectivity. The system supports assignment management from creation to the release of moderated results.

Online payments, video conferencing, automated grading, plagiarism detection and web or mobile interfaces were excluded from the project scope.

2. Phase 1: Project Planning and Management

The project followed the Waterfall model, supported by V-Model verification and validation. Planning included feasibility assessment, work breakdown, scheduling and risk management.

Technical feasibility was supported by C++17 and SQLite, which allow the system to operate without a dedicated database server. Operational feasibility considered the school's limited computing resources and users' computer skills. Economic feasibility benefited from open-source tools, while legal and schedule feasibility considered data protection and the available development period.

A Work Breakdown Structure (WBS) divided the project into manageable activities. A Gantt chart illustrated the seven-week schedule, dependencies, milestones and critical path. The risk register identified technical, project and people risks, including development delays, security vulnerabilities, integration problems and requirements inconsistencies. Mitigation measures included modular development, testing, version control and documentation.

Original Diagrams: Work Breakdown Structure | Gantt Chart

3. Phase 2: Requirements Engineering

Requirements were analysed from statements provided by the Principal, Lecturer, HoD, Student Representative and IT Officer. Conflicts were identified and resolved through agreed business rules.

The main conflict concerned mark visibility. Lecturers preferred immediate release, while the Principal and HoD required moderation. The adopted rule allows students to view marks only after HoD approval and release.

Another conflict concerned late work. The agreed policy applies a penalty of 10% of the available mark per day

for up to three days, with a zero-mark outcome beyond three days. Lecturers may grant individual extensions, and students may resubmit before the applicable deadline.

The Software Requirements Specification (SRS) documented at least 12 functional and six non-functional requirements, prioritised using MoSCoW and expressed in verifiable terms.

Key functional requirements:

FR1 – Assignment Submission: Students must be able to submit and resubmit assignments before the applicable deadline, while preserving previous attempts.

FR2 – Marking and Moderation: Lecturers must mark submissions and provide feedback, while HoDs must review marks before their release.

Key non-functional requirements:

NFR1 – Security: The system must enforce role-based access, store salted password hashes and use parameterised SQL queries.

NFR2 – Reliability and Performance: The system must preserve records between sessions, handle invalid input and support the school's expected user population.

The use-case model represents the four actors and their permitted operations. Fully dressed use-case specifications describe preconditions, main flows, alternative flows, exceptions and postconditions.

Original Diagram: Use Case Diagram

4. Phase 3: System Design

4.1 Architecture and Class Design

A layered architecture was selected, separating presentation, service, domain and data-access responsibilities. This improves maintainability and testability compared with a tightly coupled architecture.

The class design uses an abstract `User` class with inheritance for Administrator, Lecturer, Student and HeadOfDepartment. The Factory pattern supports user creation, the Strategy pattern handles late-penalty rules, and the Repository pattern separates database operations from business logic. These decisions support object-oriented design and SOLID principles.

Original Diagrams: Component Diagram | Class Diagram

4.2 Behavioural Design

The submission state machine represents the submission life cycle. The activity diagram shows the end-to-end workflow, while sequence diagrams describe submission, late and resubmission handling, and mark moderation and release.

Original Diagrams: State Machine | Activity Diagram | Submit Assignment Sequence | Moderate and Release Sequence

4.3 Database Design

SQLite provides persistent storage. The implemented database contains ten tables: `departments`, `users`, `courses`, `enrollments`, `assignments`, `extensions`, `submissions`, `marks`, `moderation_decisions` and `mark_audit_log`.

Primary keys, foreign keys and constraints maintain data integrity. The database design reduces redundancy and supports Third Normal Form. The data dictionary and SQL schema describe the tables, attributes, relationships and constraints.

Original Diagram: Entity Relationship Diagram

5. Phase 4: Implementation

The system was implemented in C++17 using CMake, SQLite and libsodium.

The Administrator manages accounts, departments, courses and enrolments. Lecturers create assignments, grant individual extensions, mark submissions and send marks for moderation. Students submit and resubmit assignments and view released results. HoDs review, approve, release or return marks for corrections.

SQLite operations are handled through the data-access layer using parameterised queries. Passwords are protected using salted Argon2id hashes through libsodium. Input validation and exception handling reduce failures caused by invalid input or database errors.

The system preserves submission history and maintains an audit log recording who changed a mark, when it changed, the previous and new values, and the reason.

Design changes: During implementation, the database was extended with department and moderation information. The final system uses `moderation_decisions` for HoD decisions and `mark_audit_log` for mark changes. Some operations originally represented in role classes were moved into service classes. The moderation workflow was also refined to support returning marks for corrections.

The original diagrams remain unchanged as the submitted design baseline. Differences are explained in the As-Built Deviation Log.

6. Phase 5: Verification and Validation

Testing followed the V-Model approach, covering unit, integration, system and acceptance testing. Test cases were derived from requirements and use cases, including valid inputs, invalid inputs, deadline boundaries and unauthorised operations.

Unit tests covered password hashing, validation, dates, penalties and marking rules. Integration tests checked interactions between services and SQLite. System tests covered complete workflows, including submission, marking, moderation and result release.

Automated testing results:

Total tests: **104**

Passed: **104**

Failed: **0**

Service-layer line coverage: **96.7%**

Service-layer branch coverage: **58.9%**

Functions covered: **54 of 54**

The service-layer line coverage exceeded the assignment's 70% target. However, branch coverage indicates that some decision paths still require further testing.

One high-severity defect, DEF-01, concerned approved marks not appearing in students' results. After correction, manual retesting confirmed that approved and released marks and feedback became visible to students.

Testing evidence, coverage results and defect details are provided in the Phase 5 documentation.

7. Phase 6: Configuration Management and Quality Assurance

Git and GitHub were used for configuration management. GitHub Actions was configured to build and test the project automatically on pushes and pull requests. The documented CI execution completed successfully with all 104 tests passing.

Cppcheck 2.22.0 analysed 32 source files and reported 23 findings: 22 style suggestions and one performance

suggestion, with no errors or warnings. The suggestions were reviewed and documented rather than claimed as fixed.

Lizard analysed 69 files and 356 functions. The average cyclomatic complexity was 2.6. Two functions exceeded the threshold of 15: `toString` in `Enums.cpp` (18) and `main` in `main.cpp` (16). These were identified for future refactoring.

Quality assurance also considered modularity, access control, input validation, secure passwords, database integrity, auditing and automated testing.

Evidence: GitHub Repository and CI

8. Phase 7: Deployment

The LMS is designed for offline deployment on a school computer. The UML deployment diagram shows the application, local SQLite database, submission storage and log file.

Installation requires the C++17 development tools, CMake, SQLite and libsodium. The technician obtains the source code, installs dependencies, configures the project, builds the application and runs the tests before initial setup.

On first use, an Administrator account is created. The Administrator then configures departments, users, courses and enrolments.

Separate user manuals explain the operations available to the Administrator, Lecturer, Student and HoD. The deployment documentation also describes backups, file permissions, initial configuration and release v1.0.

Original Diagram: UML Deployment Diagram

9. Phase 8: Software Maintenance

Change request CR1 requires lecturers to extend assignment deadlines for an entire class and display the updated deadline on students' assignment lists.

The submitted maintenance analysis classifies this as **perfective maintenance**, because it improves the system by adding useful functionality.

The proposed implementation introduces a `class_extensions` table and updates deadline calculations, assignment displays, affected requirements and relevant tests. The effective deadline would use an individual extension where applicable, otherwise a class-wide extension, or the original deadline.

The change was not implemented in version 1.0 and remains deferred. The 104 passing tests provide a regression baseline, but additional CR1 tests must be implemented and executed before release v1.1. Therefore, **version 1.1 has not been released**.

10. Professional, Legal and Ethical Issues

The LMS handles learners' personal information and academic results, requiring compliance with applicable data-protection obligations in Lesotho. Access must be restricted to authorised users, and assessment records must be protected against unauthorised disclosure or modification.

Password hashing, role-based access control and mark auditing support security and accountability. Database backups must also be protected.

Accessibility requires clear instructions, readable messages and simple navigation. Fairness requires consistent application of late penalties and extensions, with moderation and audit records supporting transparent assessment decisions.

Professional responsibilities include accurate documentation, secure development, testing changes before release and correcting identified defects.

11. Conclusion

The project produced a functional offline LMS supporting course management, assignment submission, marking, moderation and controlled release of results. It demonstrates the use of software engineering processes, object-oriented design, design patterns, persistent storage and security controls.

All 104 automated tests passed, and service-layer line coverage reached 96.7%. The project also highlighted the importance of traceability and consistency between requirements, design and implementation.

Remaining work includes implementing CR1, improving branch coverage and addressing selected code-quality recommendations.

References

National University of Lesotho. (2026). *CS4433 Software Engineering: Individual Assignment*. Department of Mathematics and Computer Science.

CS4433 LMS Project. (2026). *Project Management Plan, Requirements Engineering Documents, Software Design Document and Implementation Documentation*. Project repository.

CS4433 LMS Project. (2026). *Verification and Validation, Configuration Management and Quality Assurance, Deployment, and Software Maintenance Documentation*. Project repository.

CS4433 LMS Project. (2026). *Offline Moderated Learning Management System: Source Code, Tests and Technical Documentation*. https://github.com/makoakhotso-oss/CS4433-MODERATED-LMS-PROJECT

Supporting Evidence: The complete phase documents, original diagrams, source code, testing evidence, coverage results, user manuals and requirements traceability information are provided separately in the GitHub repository.